A PROJECT REPORT

on

“The Daily Chronicle”

Submitted by

Mr. Ankit Rajesh Yadav

Seat no.

in partial fulfillment for the award of the degree

of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

under the guidance of

Prof. Srimathi Narayanan

Department of Computer Science

VIDYAVARDHINI’S

A. V. COLLEGE OF ARTS, K. M. COLLEGE OF COMMERCE,

E. S. A. COLLEGE OF SCIENCE, VASAI(WEST),
PALGHAR-401202,

(Affiliated To University Of Mumbai)
MAHARASHTRA
(SEM-V)
(2025 - 2026)

DECLARATION

I, **Ankit Rajesh Yadav** hereby declare that the project entitled **"The Daily Chronicle"** submitted in the partial fulfilment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2025 – 2026** is my original work and the project has not formed the basis for the award of any degree, associate ship, fellowship or any other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

I would like to acknowledge my sincere thanks towards our project guide Head of Computer Science Department **Prof. Srimathi Narayanan** for her valuable guidance and suggestions and providing me an opportunity to do the project work in the college lab and which made me complete the project successfully.

I am a very thankful to my project guide **Prof. Srimathi Narayanan** for providing such nice guidance in form of comments and corrections. I am thankful to and fortunate enough to get constant encouragement, support and guidance from all teaching staff of Computer Science which helped us in successfully completing our project work. Also, I would like to extend our sincere esteems to all staffs in laboratory for their timely support.

-Ankit Rajesh Yadav

T.Y.BSC (Computer Science)

PLAGARISM REPORT

SmallSEOTools 09/18/2025 **Page 1 of 8**

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS

Plagiarism	0%	Partial Match	0%
Exact Match	0%	Unique	100%

Scan details

Total Words	Total Characters	Plagiarized Sentences	Unique Sentences
475	5064	0	1 (100%)

#1 100% Unique

```
// ================= Imports for Firestore =================

import { db } from "./firebase-config.js";

import { collection, getDocs, orderBy, query, where } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
```

GANTT CHART

INDEX

Sr.no	Contents	Page no	Sign
1.	**Introduction and Objective**	6	
2.	**Scope of Project and Technology stack**	7	
3.	**Feasibility Study**	9	
4.	**System Features and Modules**	10	
5.	**Use case and Advantages**	12	
6.	**Hardware and Software Requirement**	14	
7.	**System Design Details** 1. Event Table 2. Entity Relationship Diagram 3. Class Diagram 4. Activity Diagram 5. Use Case Diagram 6. Sequence Diagram 7. Component Diagram 8. Deployment Diagram 9. Database Model	 16 17 18 19 20 21 22 23 24	
8.	**System Implementation (CODE)**	25	
9.	**User Interface (Screenshots)**	37	
10.	**References**	41	

INTRODUCTION AND OBJECTIVE

Introduction

The Daily Chronicle is a dynamic and interactive web-based news portal platform designed to deliver real-time news to users while also empowering editors to contribute their own articles. The portal integrates the **NewsAPI** for fetching the latest news from global sources and **Firebase Firestore** for storing editor-added articles. These editor-generated articles are displayed in a dedicated "Recently Added" category, ensuring that both global and local perspectives are represented on the platform.

The system enhances user engagement by offering **category-based navigation, date filters**, thereby improving accessibility and personalization. The user interface is developed using **HTML, CSS, and JavaScript**, ensuring a responsive and seamless browsing experience across devices. Editors are provided with a dashboard where they can add articles by specifying the **title, description, and image URL**, which are then published in real-time for users.

By bridging API-driven international news with editor-managed local content, this project creates a powerful and user-friendly platform that not only keeps users informed but also encourages active participation from content creators.

Objective

The main objectives of the News Portal Website with Editor Dashboard **(The Daily Chronicle)** are to:

- **Provide real-time news** by integrating the NewsAPI, ensuring users stay updated with global headlines.
- **Enable editors to contribute** their own articles, stored securely in Firebase Firestore and displayed under "Recently Added."
- **Implement filtering options** (e.g., by date) to allow users to browse news articles more efficiently.
- **Design an interactive editor dashboard** with an intuitive interface for submitting news articles.
- **Ensure a responsive user interface** that works seamlessly across desktops, tablets, and mobile devices.
- **Promote engagement** by combining automated API news with human-curated editor contributions.
- **Lay the foundation for scalability**, enabling future enhancements such as AI-driven recommendations, user login, and commenting features.

SCOPE OF PROJECT AND TECHNOLOGY STACK

Scope

The **The Daily Chronicle** aims to provide a comprehensive, interactive, and scalable platform for delivering real-time news while enabling editors to publish their own content. Its scope includes:

- **News Categories:** Users can browse news across categories such as Technology, Business, Sports, Entertainment, and Health, fetched dynamically through the NewsAPI.
- **Recently Added Section:** Editors can contribute their own articles, which are stored in Firebase Firestore and displayed in a dedicated "Recently Added" category.
- **Date-Based Filters:** Users can filter articles by publication date (e.g., today, last 7 days, last 30 days) to quickly access relevant information.
- **Search Functionality:** Users can perform keyword-based searches to find specific articles more efficiently.
- **Responsive User Interface:** The platform is optimized for desktops, tablets, and mobile devices, ensuring accessibility for a wide audience.
- **Cross-Platform Availability:** Since it is built with web technologies, the portal can be accessed from any modern browser without installation.
- **Scalability and Future Enhancements:** The project can be expanded to include user authentication, commenting systems, push notifications, and AI-powered personalized news recommendations.

Technology Stack:

The project is implemented using the following technologies:

Frontend (User Interface):

- **HTML5 & CSS3:** Used for designing the structure and styling of the news portal, ensuring responsiveness and visual appeal.
- **JavaScript (ES6):** Implements interactivity, API integration, filters functionality.

Backend & Database:

- **Firebase Firestore:** A NoSQL cloud database used to store editor-contributed news articles in collections with fields like title, description, image URL, and timestamp.
- **Firebase Authentication :**is integrated to manage secure editor logins.

API Integration:

- **NewsAPI:** Provides real-time global news data across categories, which is fetched and displayed on the portal.

Cloud Storage (Optional for Extensions):

- **Firebase Storage:** Can be used to store images uploaded by editors for their news articles.

Additional Features & Libraries:

- **Search & Filters:** Dynamic filtering and keyword-based search functionalities implemented in JavaScript.
- **Responsive Design:** Media queries in CSS ensure compatibility across devices.

Feasibility Study

1. Technical Feasibility

- The system uses modern and reliable technologies like **Firebase Firestore**, **Firebase Authentication**, and **NewsAPI**, which are proven to be scalable and secure.
- Development can be done with **ReactJS (frontend)** and **Firebase backend**, which reduces complexity as both are cloud-ready and easy to integrate.
- Hosting is feasible on platforms like **Firebase Hosting** or **Vercel**, ensuring minimal downtime and smooth deployment.

2. Economic Feasibility

- Firebase provides a free tier with sufficient limits for small to medium-scale projects, making the cost of development low.
- NewsAPI also has a free/affordable plan, reducing the budget requirements.
- The cost-to-benefit ratio is high since the system provides a wide audience reach, advertisement opportunities, and future scalability with minimal investment.

3. Operational Feasibility

- Editors can easily log in and upload articles through a simple UI, reducing training requirements.
- Readers can browse news in real-time with features like **search, filter, and recently added sections**, improving user experience.
- System maintenance is minimal because Firebase handles authentication, database, and hosting automatically.

4. Schedule Feasibility

- Since the project uses ready-to-use services like Firebase Authentication, Firestore, and NewsAPI, development time is significantly reduced.
- A basic MVP (Minimum Viable Product) can be completed in **8–12 weeks**, while advanced features like AI recommendations or comments can be added later.
- Agile methodology can be used to break the project into **sprints**, ensuring steady progress.

5. Social Feasibility

- The platform benefits both editors and readers by merging **global news** with **local contributions**.
- Users get easy access to authentic news, increasing trust and reliability.
- The project also encourages **digital literacy** and provides a single platform for accessing diverse categories of news.

SYSTEM FEATURES AND MODULES

System Features

1. **Editor Authentication**: Secure login of Editors using Firebase Authentication.

2. **Editor Dashboard:** Provides a secure interface where editors can add news articles with title, description, and image URL.

3. **Recently Added Section:** Displays editor-contributed news articles stored in Firebase Firestore.

4. **API Integration:** Fetches real-time news articles from NewsAPI across multiple categories such as Technology, Sports, Entertainment, and Business.

5. **Search Functionality:** Allows users to search for specific news articles using keywords.

6. **Date Filter:** Enables users to filter news by publication dates (Today, Last 7 Days, Last 30 Days).

7. **Responsive Design:** Ensures the portal works seamlessly across desktops, tablets, and mobile devices.

8. **Interactive UI:** News is displayed in visually appealing cards with images, headlines, and descriptions.

9. **News Details Page:** Clicking on a card redirects users to a detailed page with full content and related news fetched from NewsAPI.

10. **Secure Content Management:** Firebase Firestore ensures secure storage and retrieval of editor-submitted news articles.

Modules

1. **Authentication Module** –
   This module manages the secure login process for editors using Firebase Authentication. It ensures that only authorized editors can access the dashboard and contribute articles to the platform.

2. **Editor Module** –
   The editor module provides an interface for editors to add articles by entering a title, description, and image URL. It validates the inputs and then sends the data to Firestore for storage.

3. **Recently Added Module** –
   This module displays articles contributed by editors in the "Recently Added" category. It fetches data from Firestore and ensures that newly uploaded content is immediately visible to readers.

4. **API News Module** –
   The API news module integrates with the external NewsAPI to fetch real-time news articles. It ensures that the portal provides up-to-date information across categories like Politics, Sports, Education, and Technology.

5. **Search Module** –
   This module allows users to perform keyword-based searches across both API-sourced news and editor-uploaded articles. It improves content accessibility by delivering relevant results quickly.

6. **Filter Module** –
   The filter module enables date-based browsing of news articles. It helps readers narrow down articles based on publishing dates for efficient content discovery.

7. **News Details Module** –
   This module opens a detailed view of any selected article on a new page. Along with the full article, it also fetches and displays related stories for improved engagement.

8. **Responsive UI Module** –
   The responsive UI module ensures that the portal works seamlessly on desktops, tablets, and smartphones. It adjusts layouts and components dynamically for different screen sizes.

9. **Database Module** –
   This module manages all Firestore operations, including adding, retrieving, and updating editor-contributed news. It ensures secure and efficient data storage and retrieval.

10. **Scalability Module (Future Scope)** –
    This placeholder module is designed for future improvements such as user login, commenting, and push notifications. It ensures the system is flexible and capable of scaling as requirements grow.

USECASE AND ADVANTAGE

Title: Browsing and Contributing News Articles

1. **Actors:**
   - User (reader)
   - Editor (content contributor)
   - Admin (system manager – optional)
2. **Preconditions:**
   - The user has an internet connection.
   - The editor has access to the editor dashboard.
3. **Main Flow:**
   - The user opens the portal and views categories of news fetched from the NewsAPI.
   - The user browses news cards and clicks on an article to view full details.
   - The editor logs into the dashboard and submits a new article with a title, description, and image.
   - The submitted article is stored in Firebase Firestore and displayed in the "Recently Added" section.
   - The user applies date filters or uses the search bar to find relevant news.
   - The system dynamically updates the content.
4. **Postconditions:**
   - Users successfully browse and read news articles.
   - Editor-contributed articles appear in the "Recently Added" section in real time.

Advantages of the News Portal Website (The Daily Chronicle)

1. **Real-Time Updates –**
   Through integration with the NewsAPI, users can access breaking news and global updates instantly. This ensures the platform remains dynamic and relevant, giving readers up-to-date information without delays.

2. **Local Content Contribution –**
   Editors can add their own news articles, bridging the gap between international headlines and community-focused stories. This makes the platform useful not only for global awareness but also for covering local events and issues.

3. **Efficient Browsing –**
   The inclusion of search and filter options enables readers to quickly find articles based on keywords or dates. This reduces the effort required to navigate large volumes of news and makes the browsing experience smoother.

4. **Interactive UI –**
   A card-based design presents articles in a structured, visually appealing way. This interactive layout enhances readability and encourages users to engage more with the content.

5. **Cross-Platform Access –**
   Since the portal runs on any modern browser, it is accessible across desktops, laptops, tablets, and smartphones. Users do not need to install additional software, ensuring wider reach and convenience.

6. **Secure Storage –**
   Editor-contributed news is stored in Firebase Firestore, which provides encrypted, reliable, and scalable cloud storage. This ensures that articles remain safe and can be retrieved at any time.

7. **Scalability –**
   The system design allows for future expansion, such as adding user accounts, comments, or AI-driven recommendations. This ensures the project can grow in functionality without major redesign.

8. **User Engagement –**
   By combining news fetched from APIs with editor-uploaded content, the portal provides variety and depth. This mix keeps readers engaged and encourages them to return regularly for fresh and diverse content.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

1. For Development

- **Processor:** Intel Core i5 or higher (or equivalent AMD processor)
- **RAM:** Minimum 8 GB (16 GB recommended for smooth performance during development)
- **Storage:** At least 10 GB free space for project files, dependencies, and Firebase SDKs
- **Operating System:**
  - Windows 10 or higher
  - macOS 10.14 or higher
- **Screen Resolution:** 1366 × 768 or higher (1920 × 1080 recommended for development)
- **Internet Connection:** Stable broadband connection for API and Firebase integration

2. For Users (End Users of the Website)

- **Device:** Desktop, Laptop, Tablet, or Smartphone with a modern web browser
- **Processor:** Minimum dual-core (quad-core or higher recommended)
- **RAM:** 4 GB or more for smooth browsing
- **Operating System:**
  - Windows 8/10/11
  - macOS 10.14 or higher
  - Android 7.0+ or iOS 12+ (for mobile browsers)
- **Internet Connection:** Required for fetching API-based news and retrieving editor-added content from Firestore

Software Requirements

1. For Development

- **Languages & Technologies:**
  - HTML5, CSS3, JavaScript (ES6)
- **Frameworks & Libraries:**
  - Bootstrap / Custom CSS for responsive design (optional)
- **Backend & Database:**
  - Firebase Firestore (NoSQL cloud database)
  - Firebase Storage (for image uploads – optional)
- **APIs:**
  - NewsAPI (for fetching real-time global news)
- **Development Tools:**
  - Visual Studio Code (recommended IDE)
- **Version Control:**
  - Git / GitHub (for collaboration and version tracking)
- **Testing Tools:**
  - Chrome DevTools / Firefox Developer Tools for debugging and responsive testing

2. For End Users

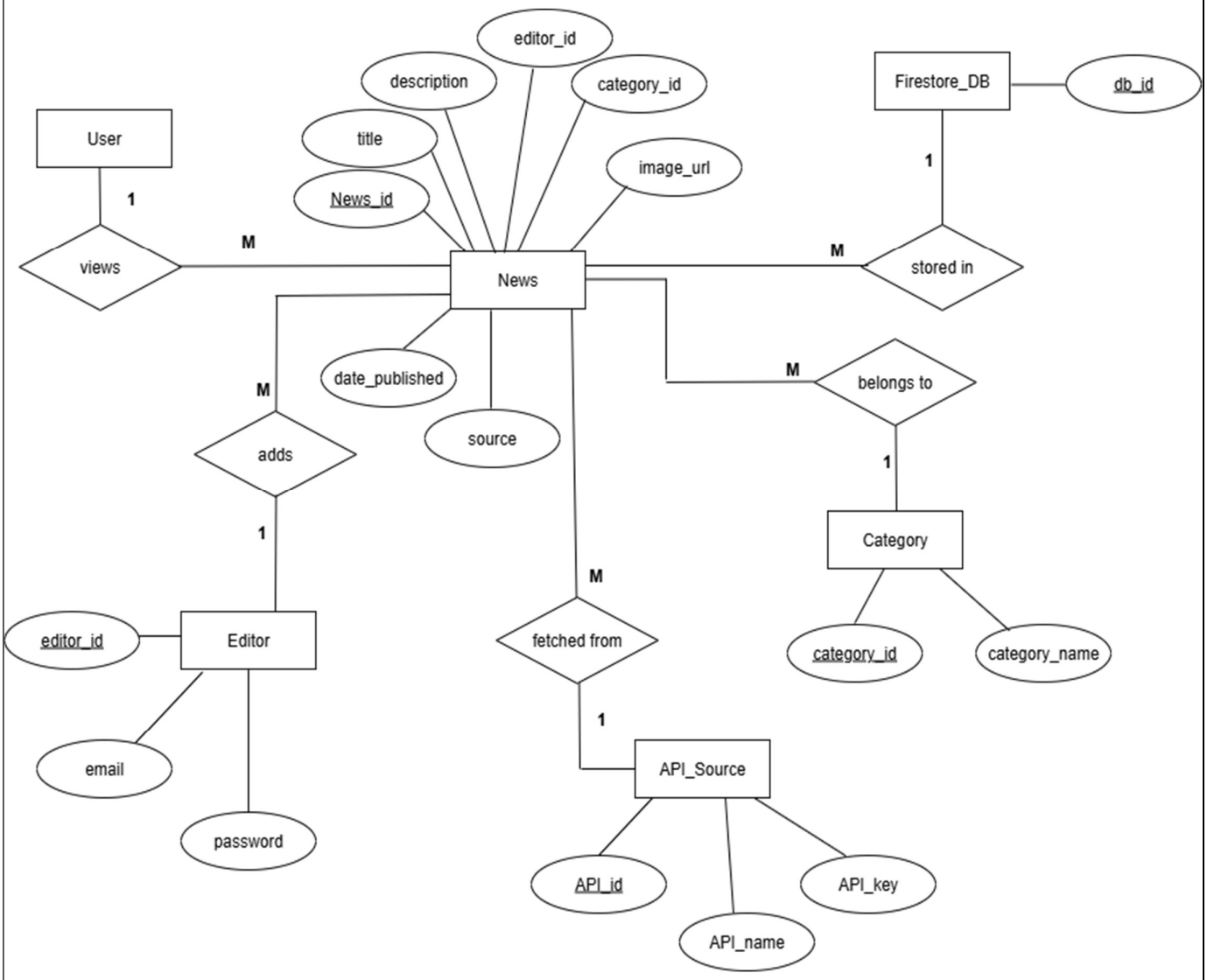
- **Operating System:**
  - Windows, macOS, Linux, Android, or iOS
- **Web Browser:**
  - Google Chrome (latest version)
  - Mozilla Firefox (latest version)
  - Microsoft Edge
- **Internet Requirement:**
  - Active internet connection to view API-based news and Firestore content

SYSTEM DESIGN

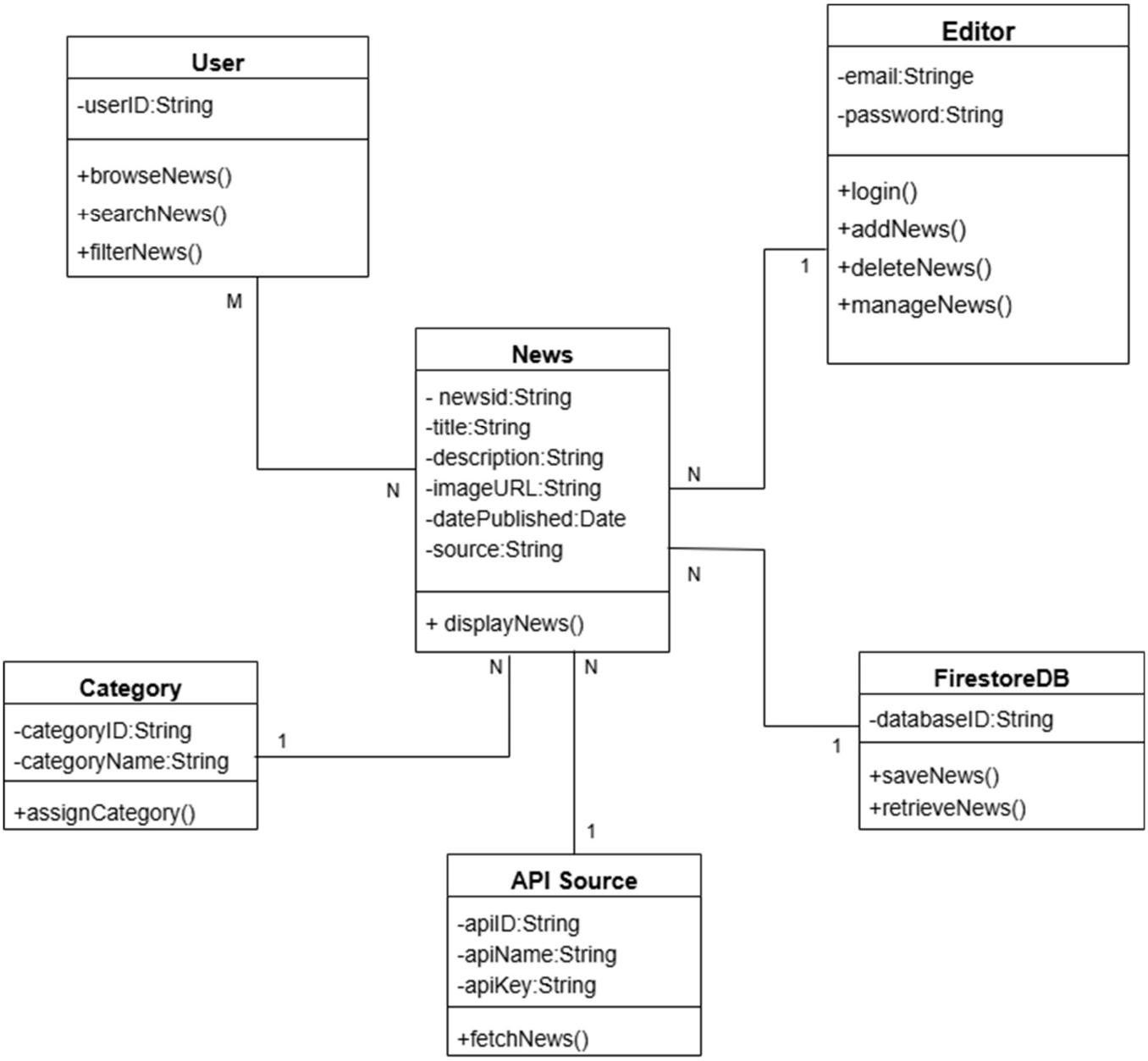
1. Event Table

Sr. no	Event	Trigger	Source	Activity	Response	Destination
1	User opens the website	User navigates to home page	User	System sends request to NewsAPI	Loads home page and fetches latest news	Browser UI
2	Browse category	User clicks a category	User	API call with selected category	Displays category-specific news	Browser UI
3	Search news	User enters keywords in search bar	User	System queries NewsAPI & Firestore	Displays matching articles	Browser UI
4	Apply date filter	User selects a date range	User	Filter applied to fetched dataset	Shows news filtered by date	Browser UI
5	Open news details	User clicks on a news card	User	System fetches article content & related news	Displays full article and related stories	Browser UI
6	Editor authentic-ation	Editor enters login credentials	Editor	Credentials verified against Firebase Auth	Grants access to editor dashboard	Editor Dashboard
7	Add news article	Editor submits title, description, image URL	Editor	Data sent to Firestore	Stores article in DB and updates "Recently Added"	Firestore DB → Browser UI
8	Database update	New article added by editor	Fire-store	Triggers real-time update listener	Updates "Recently Added" section automatically	Browser UI
9	View recently added	User selects "Recently Added" category	User	System queries Firestore collection	System queries Firestore collection	Browser UI
10	Fetch related news	User opens a news detail page	System auto-trigger	Extracts keywords from article title	Fetches & displays related articles from NewsAPI	Browser UI

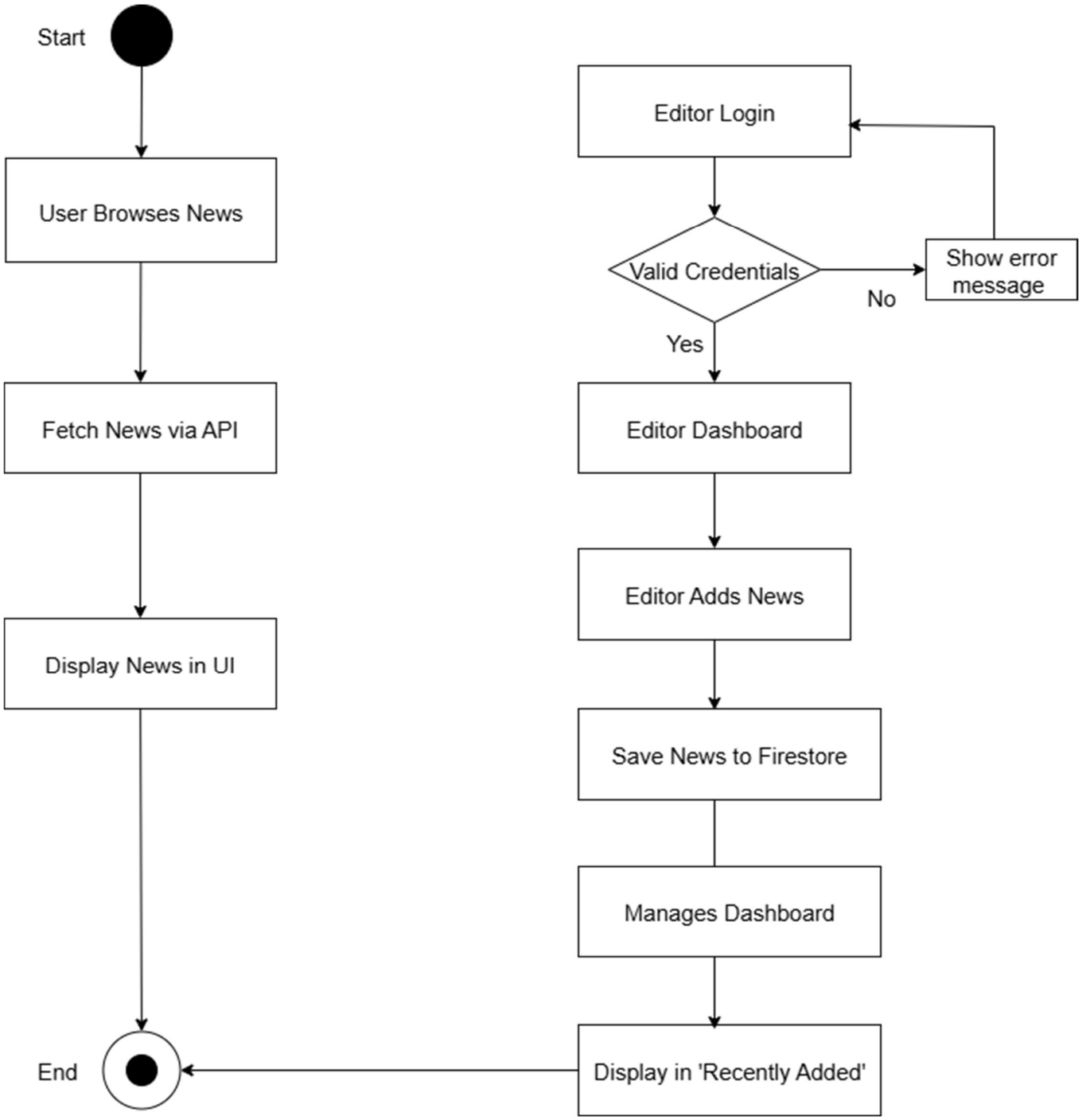
2. ENTITY-RELATIONSHIP DIAGRAM



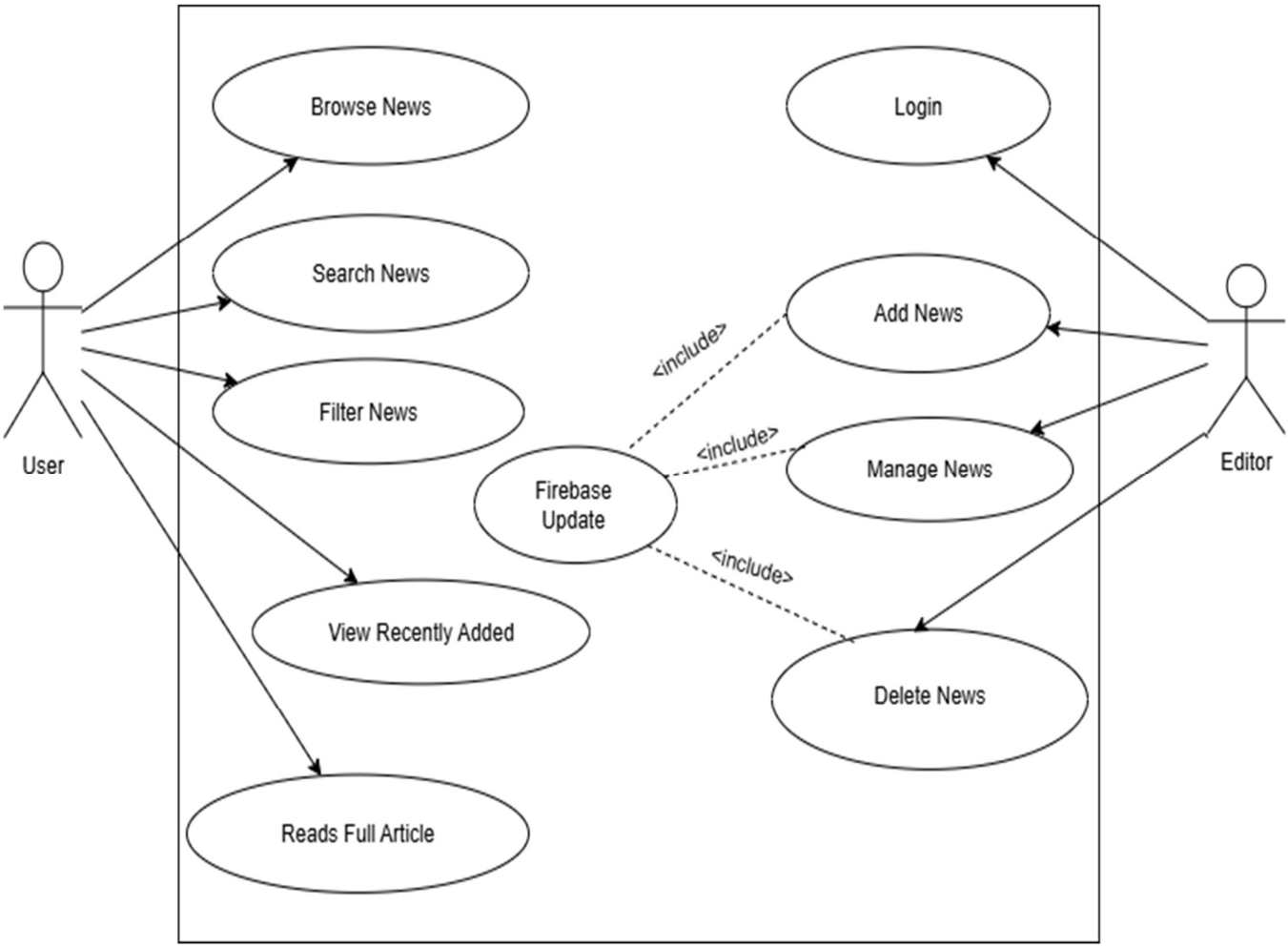
3. CLASS DIAGRAM



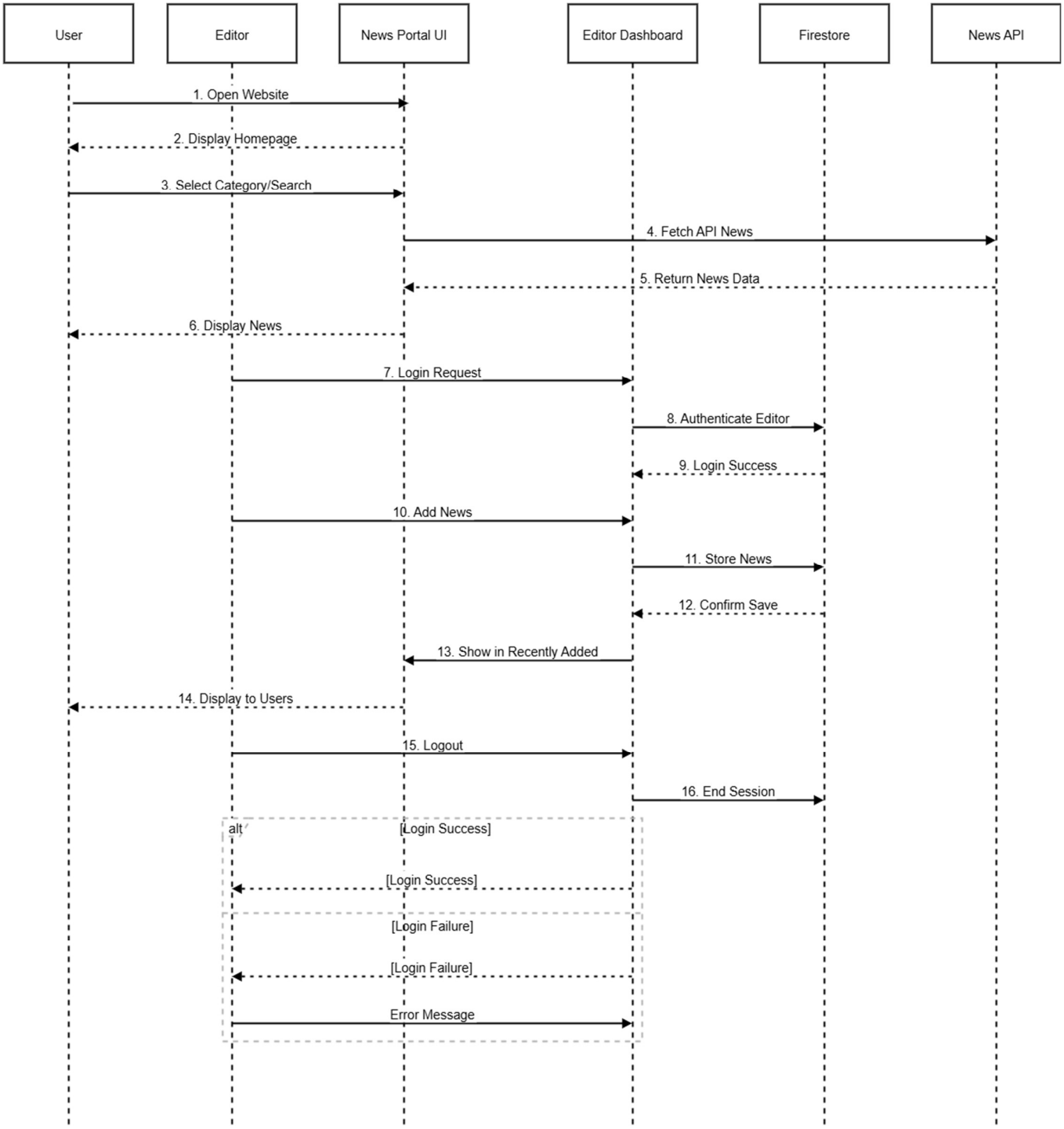
4. ACTIVITY DIAGRAM



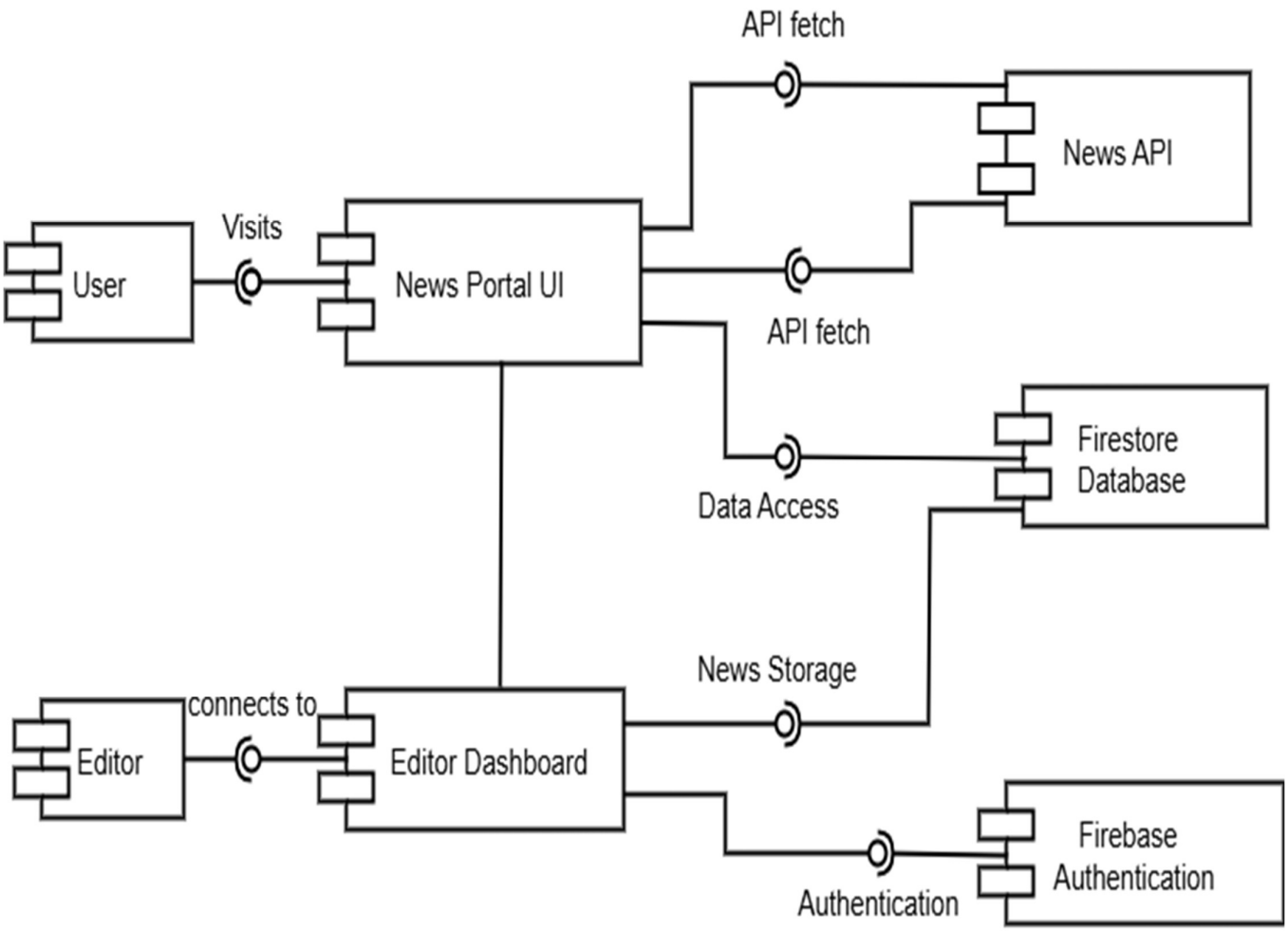
5. USECASE DIAGRAM



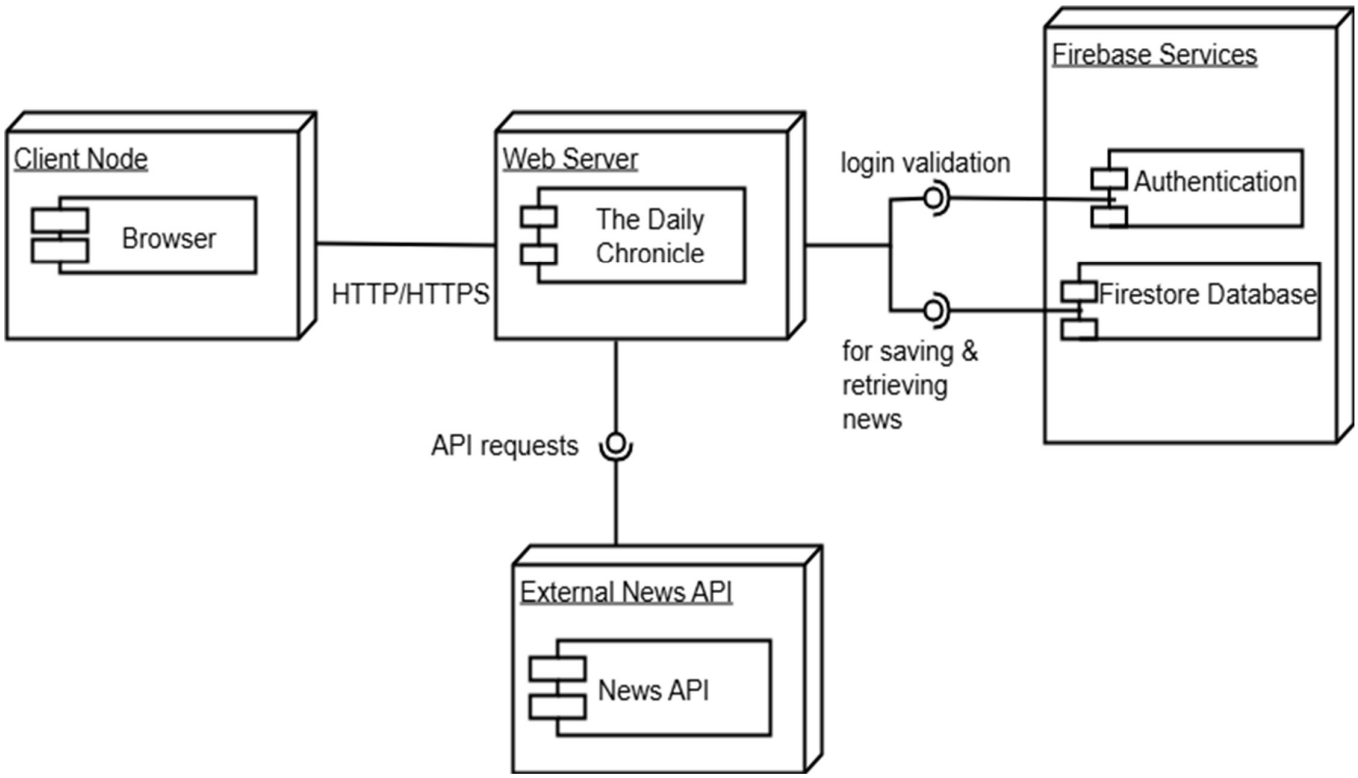
6. SEQUENCE DIAGRAM



7. COMPONENT DIAGRAM



8. DEPLOYMENT DIAGRAM



9. DATABASE MODEL

Editor

Field	Datatype	Key
editor_id	String	PK
name	String	
password_hash	String	

NewsArticles Table

Field	Datatype	Key
article_id	String	PK
title	String	
description	String	
image URL	String (URL)	
Category	String	
Created_at	Timestamp	

RecentlyAdded Table

Field	Datatype	Key
recent_id	String	PK
article_id	String	FK

SYTEM IMPLEMENTATION

app.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>The Daily Chronicle</title>
  <link rel="stylesheet" href="app.css">
 <script type="module" src="app.js" defer></script>
</head>
<body>
  <div class="container">

    <!-- Fixed Logo -->
    <div class="logo">
      <h1>The Daily Chronicle</h1>
      <button class="login-btn" onclick="window.location.href='login.html'">Login</button>
    </div>

    <!-- Categories + Search -->
    <nav>
      <ul class="categories">
        <li id="politics" onclick="navClick('politics')">Politics</li>
        <li id="sports" onclick="navClick('sports')">Sports</li>
        <li id="education" onclick="navClick('education')">Education</li>
        <li id="technology" onclick="navClick('technology')">Technology</li>
```

```
        <li id="entertainment" onclick="navClick('entertainment')">Entertainment</li>
         <li id="recent" onclick="navClick('recent')">Recently Added</li>


          <!-- Date Filter Dropdown -->
  <li>
    <select id="dateFilter">
      <option value="all">All Time</option>
      <option value="today">Today</option>
      <option value="7days">Last 7 Days</option>
      <option value="30days">Last 30 Days</option>
    </select>
  </li>
</ul>
      </ul>
      <div class="searchBar">
        <input type="text" placeholder="Search here" id="inputData">
        <button id="searchBtn">Search</button>
      </div>
    </nav>


    <!-- Category Title -->
    <div class="category">
      <h1 id="type"></h1>
    </div>


    <!-- Loader -->
    <div class="loader-container" id="loader">
      <div class="loader"></div>
      <p>Loading news...</p>
```

```
    </div>


    <!-- News Cards -->

    <div class="cardData"></div>


  </div>

</body>

</html>
```

app.js

```
// ================= Imports for Firestore =================

import { db } from "./firebase-config.js";

import { collection, getDocs, orderBy, query, where } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";


// ================= Global Variables =================

let key = "3c2b7c6b562b4f46ae97f1aa300da1aa"; //  API key

let cardData = document.querySelector(".cardData");

let searchBtn = document.getElementById("searchBtn");

let inputData = document.getElementById("inputData");

let searchType = document.getElementById("type");

const loader = document.getElementById("loader");


// ================= Date Filter Helper =================

function getDateFilterRange(option) {

  let today = new Date();

  if (option === "today") {

    today.setHours(0,0,0,0);

    return today;
```

```
  } else if (option === "7days") {
    today.setDate(today.getDate() - 7);
    today.setHours(0,0,0,0);
    return today;
  } else if (option === "30days") {
    today.setDate(today.getDate() - 30);
    today.setHours(0,0,0,0);
    return today;
  }
  return null; // all time
}

// ================= Fetch News from API =================
async function getData(input) {
  try {
    loader.style.display = "block";
    cardData.innerHTML = "";

    let filter = document.getElementById("dateFilter").value;
    let fromDate = getDateFilterRange(filter);

    let url = `https://newsapi.org/v2/everything?q=${input}&apiKey=${key}`;
    if (fromDate) {
      url += `&from=${fromDate.toISOString().split("T")[0]}`;
    }

    let res = await fetch(url);
    let jsonData = await res.json();

    loader.style.display = "none";
```

```
    searchType.innerText = input.charAt(0).toUpperCase() + input.slice(1);

    if (!jsonData.articles || jsonData.articles.length === 0) {
      cardData.innerHTML = "<p style='text-align:center;'>No articles found.</p>";
      return;
    }

    jsonData.articles.forEach(article => {
      let divs = document.createElement("div");
      divs.classList.add("card");

      divs.innerHTML = `
        <img src="${article.urlToImage || 'https://via.placeholder.com/400x200'}" alt="News Image">
        <h3>${article.title}</h3>
        <p>${article.description || 'No description available.'}</p>
        <small style="color:gray; margin: 0 12px 12px; display:block;">
          ${new Date(article.publishedAt).toLocaleString()}
        </small>
      `;

      divs.addEventListener("click", () => window.open(article.url, "_blank"));
      cardData.appendChild(divs);
    });
  } catch (err) {
    console.error("Error fetching API news:", err);
    loader.style.display = "none";
    cardData.innerHTML = "<p style='text-align:center;'>Failed to load API news.</p>";
  }
}
```

```
// ================= Load Recent News from Firestore =================
async function loadRecentNews() {
  cardData.innerHTML = "";
  loader.style.display = "block";
  searchType.innerText = "Recently Added";

  try {
    let filter = document.getElementById("dateFilter").value;
    let fromDate = getDateFilterRange(filter);

    let q;
    if (fromDate) {
      q = query(
        collection(db, "news"),
        where("createdAt", ">=", fromDate),
        orderBy("createdAt", "desc")
      );
    } else {
      q = query(
        collection(db, "news"),
        orderBy("createdAt", "desc")
      );
    }

    const snapshot = await getDocs(q);
    loader.style.display = "none";

    if (snapshot.empty) {
      cardData.innerHTML = "<p style='text-align:center;'>No recent articles found.</p>";
      return;
```

```
  }

  snapshot.forEach(docSnap => {
    const news = docSnap.data();
    const id = docSnap.id;

    let divs = document.createElement("div");
    divs.classList.add("card");

    divs.innerHTML = `
      <img src="${news.imageUrl || 'https://via.placeholder.com/400x200'}" alt="News Image">
      <h3>${news.title}</h3>
      <p>${news.description || ''}</p>
      <small style="color:gray; margin: 0 12px 12px; display:block;">
        Recently Added
      </small>
    `;

    divs.addEventListener("click", () => {
      window.open(`news.html?id=${id}`, "_blank");
    });

    cardData.appendChild(divs);
  });

} catch (err) {
  console.error("Error fetching Firestore news:", err);
  loader.style.display = "none";
  cardData.innerHTML = "<p style='text-align:center;'>Failed to load recent news.</p>";
}
```

```
}
window.loadRecentNews = loadRecentNews;

// ================= Default Load =================
window.addEventListener("load", () => {
  getData("India");   // Default API news
});

// ================= Search =================
searchBtn.addEventListener("click", () => {
  let text = inputData.value.trim();
  if (text) getData(text);
  else cardData.innerHTML = "<p style='text-align:center;'>Please enter a search term.</p>";
});

// ================= Navigation =================
function navClick(name) {
  document.querySelectorAll(".categories li").forEach(li => li.classList.remove("active"));
  let clicked = document.getElementById(name);
  if (clicked) clicked.classList.add("active");

  if (name === "recent") loadRecentNews();
  else getData(name);
}
window.navClick = navClick;

// ================= Filter Listener =================
document.getElementById("dateFilter").addEventListener("change", () => {
  let activeCategory = document.querySelector(".categories li.active");
  let category = activeCategory ? activeCategory.id : "India";
```

```
  if (category === "recent") {
    loadRecentNews(); // Firestore filter apply
  } else {
    getData(category); // API filter apply
  }
});
```

Dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Editor Dashboard | Daily Chronicle</title>
  <link rel="stylesheet" href="dashboard.css">
  <script type="module" src="dashboard.js" defer></script>
</head>
<body>
  <header class="navbar">
    <h2>Editor Dashboard</h2>
    <button id="logoutBtn" class="logout-btn">Logout</button>
  </header>

  <div class="dashboard">
    <!-- Sidebar -->
    <aside class="sidebar">
      <ul>
        <li class="active" data-section="addNews">➕ Add News</li>
        <li data-section="manageNews">📰 Manage News</li>
      </ul>
    </aside>

    <!-- Main content -->
    <main class="main-content">
      <!-- Add News Section -->
      <section id="addNews" class="section active">
        <h3>Add News</h3>
        <form id="newsForm" class="news-form">
```

```
        <div class="form-group">
          <label for="title"> News Title</label>
          <input type="text" id="title" placeholder="Enter news title" required>
        </div>

        <div class="form-group">
          <label for="description">News Description</label>
          <textarea id="description" placeholder="Write a short description..." required></textarea>
        </div>

        <div class="form-group">
          <label for="imageUrl">🖼 Image URL</label>
          <input type="text" id="imageUrl" placeholder="Paste image link (optional)">
        </div>

        <button type="submit">➕ Submit News</button>
      </form>
    </section>

    <!-- Manage News Section -->
    <section id="manageNews" class="section">
      <h3>Manage News</h3>
      <div id="newsList" class="news-list"></div>
    </section>
  </main>
 </div>
</body>
</html>
```

Dashboard.js

```
import { auth, db } from "./firebase-config.js";
import {
  collection, addDoc, getDocs, deleteDoc, doc, serverTimestamp, orderBy, query
} from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
import { signOut } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";

const newsForm = document.getElementById("newsForm");
const newsList = document.getElementById("newsList");
const logoutBtn = document.getElementById("logoutBtn");

// ================ Sidebar Navigation =================
document.querySelectorAll(".sidebar li").forEach(item => {
  item.addEventListener("click", () => {
    document.querySelectorAll(".sidebar li").forEach(li => li.classList.remove("active"));
```

```
    item.classList.add("active");

    let sectionId = item.getAttribute("data-section");
    document.querySelectorAll(".section").forEach(sec => sec.classList.remove("active"));
    document.getElementById(sectionId).classList.add("active");
  });
});

// ================= Add News =================
newsForm.addEventListener("submit", async (e) => {
  e.preventDefault();

  const title = document.getElementById("title").value;
  const description = document.getElementById("description").value;
  const imageUrl = document.getElementById("imageUrl").value || "https://via.placeholder.com/150";

  try {
    await addDoc(collection(db, "news"), {
      title,
      description,
      imageUrl,
      createdAt: serverTimestamp()
    });

    alert("✅ News added successfully!");
    newsForm.reset();
    loadNews();
  } catch (error) {
    console.error("Error adding news:", error);
  }
});

// ================= Load & Manage News =================
async function loadNews() {
  newsList.innerHTML = "<p>Loading...</p>";
  const q = query(collection(db, "news"), orderBy("createdAt", "desc"));
  const querySnapshot = await getDocs(q);

  newsList.innerHTML = "";

  querySnapshot.forEach((docSnap) => {
    const news = docSnap.data();

    const div = document.createElement("div");
    div.classList.add("card");
    div.innerHTML = `
      <img src="${news.imageUrl}" alt="">
      <h3>${news.title}</h3>
```

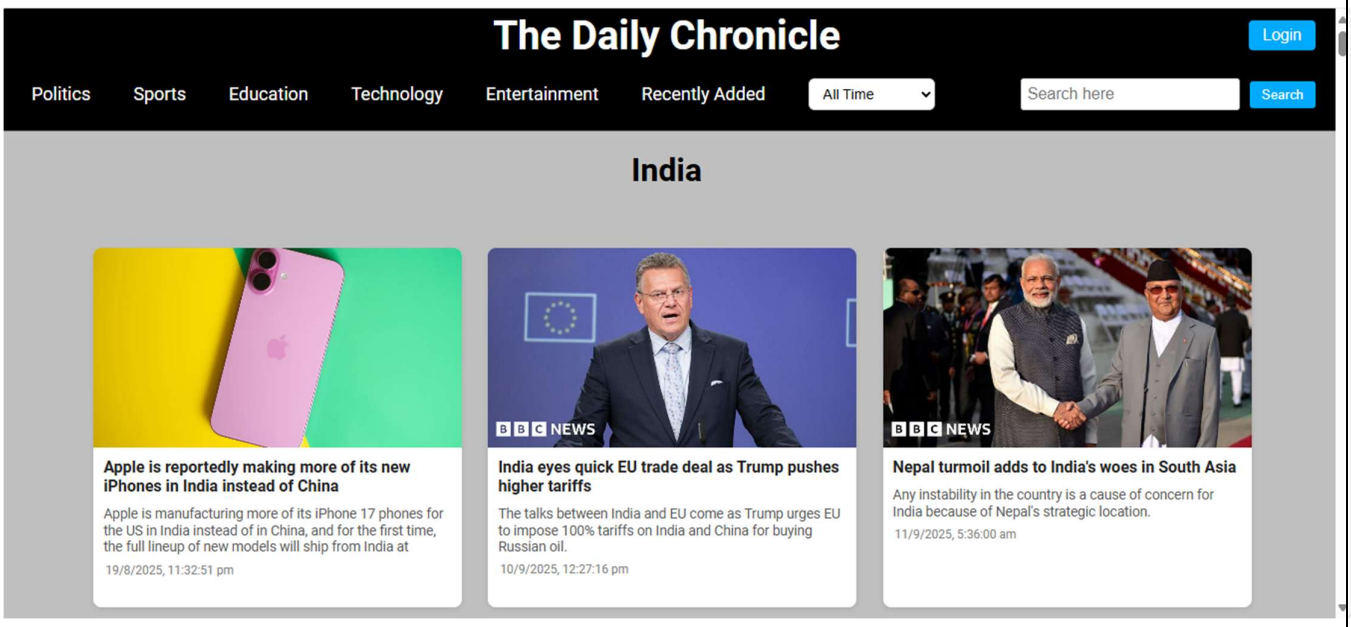
```
      <p>${news.description}</p>
      <button class="deleteBtn">Delete</button>
    `;

    // Delete news
    div.querySelector(".deleteBtn").addEventListener("click", async () => {
      await deleteDoc(doc(db, "news", docSnap.id));
      loadNews();
    });

    newsList.appendChild(div);
  });
}
loadNews();

// ================= Logout =================
logoutBtn.addEventListener("click", async () => {
  await signOut(auth);
  window.location.href = "login.html";});
```

Firebase-config.js

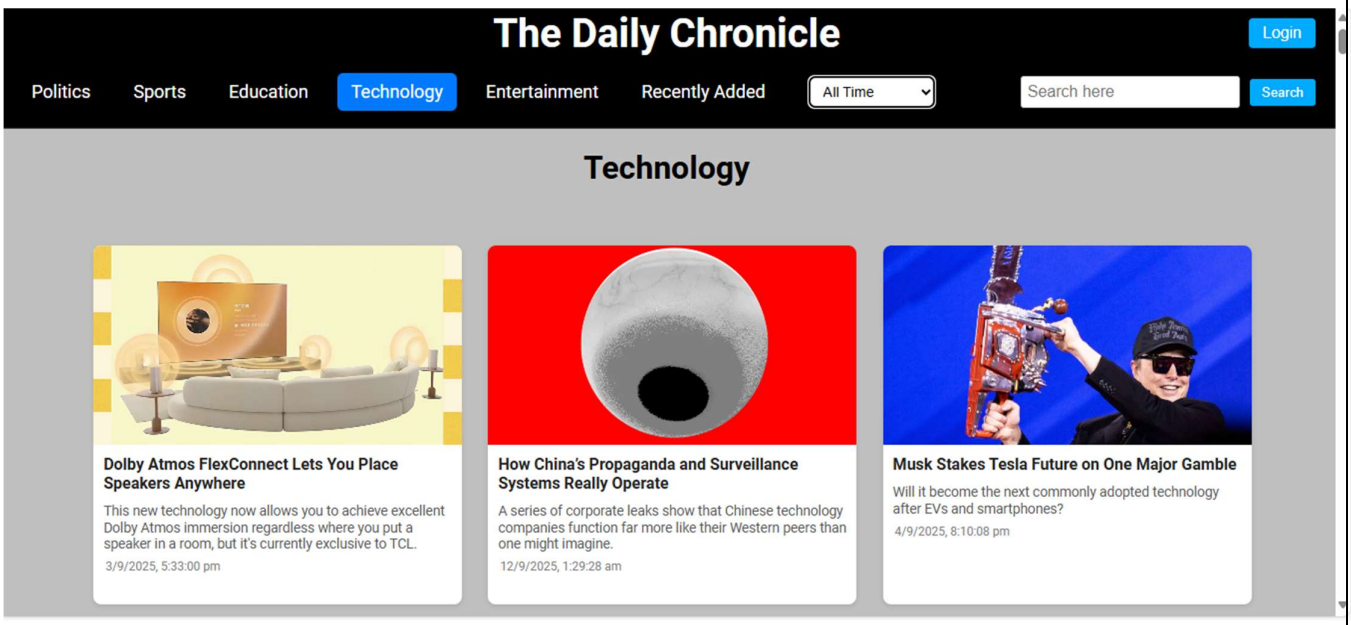
```
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";

const firebaseConfig = {
  apiKey: "AIzaSyBLWXsntvHFRmn-w_665gJexiYp7zmC-og",
  authDomain: "dailychronicle-a9ce1.firebaseapp.com",
  projectId: "dailychronicle-a9ce1",
  storageBucket: "dailychronicle-a9ce1.appspot.com",
  messagingSenderId: "896770911755",
  appId: "1:896770911755:web:08eba447dbfc2e4a57610e"
};

export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

USER INTERFACE

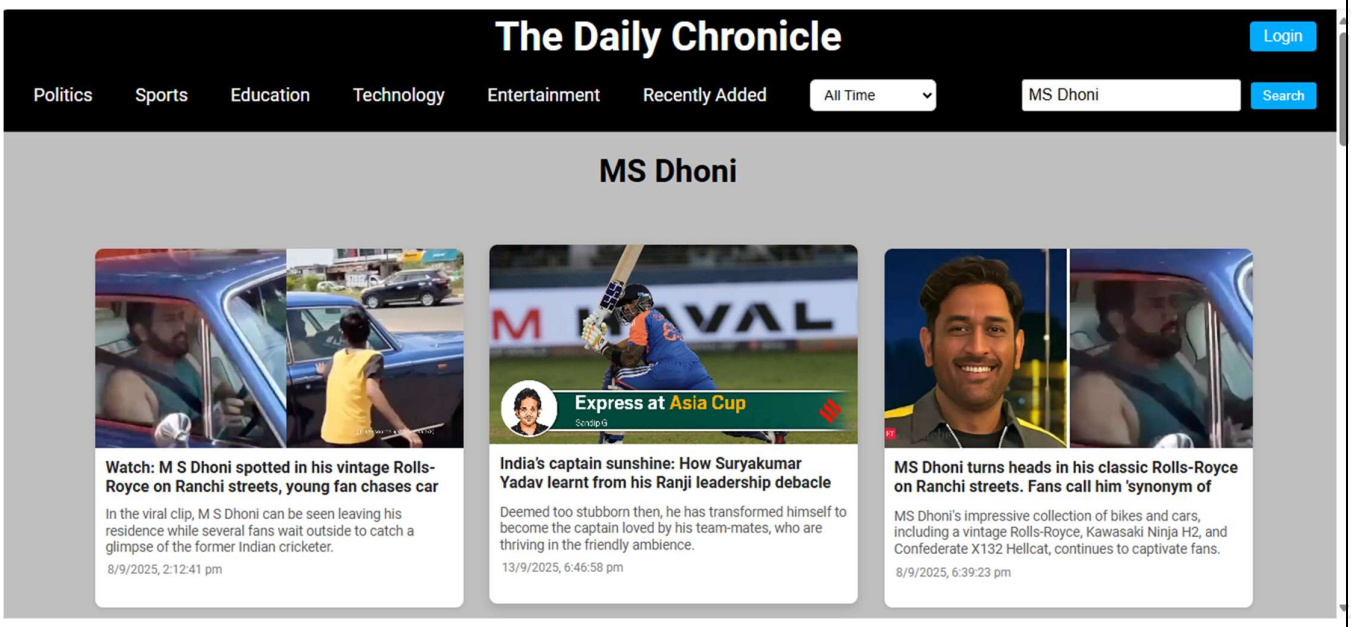
ONBOARDING SCREEN



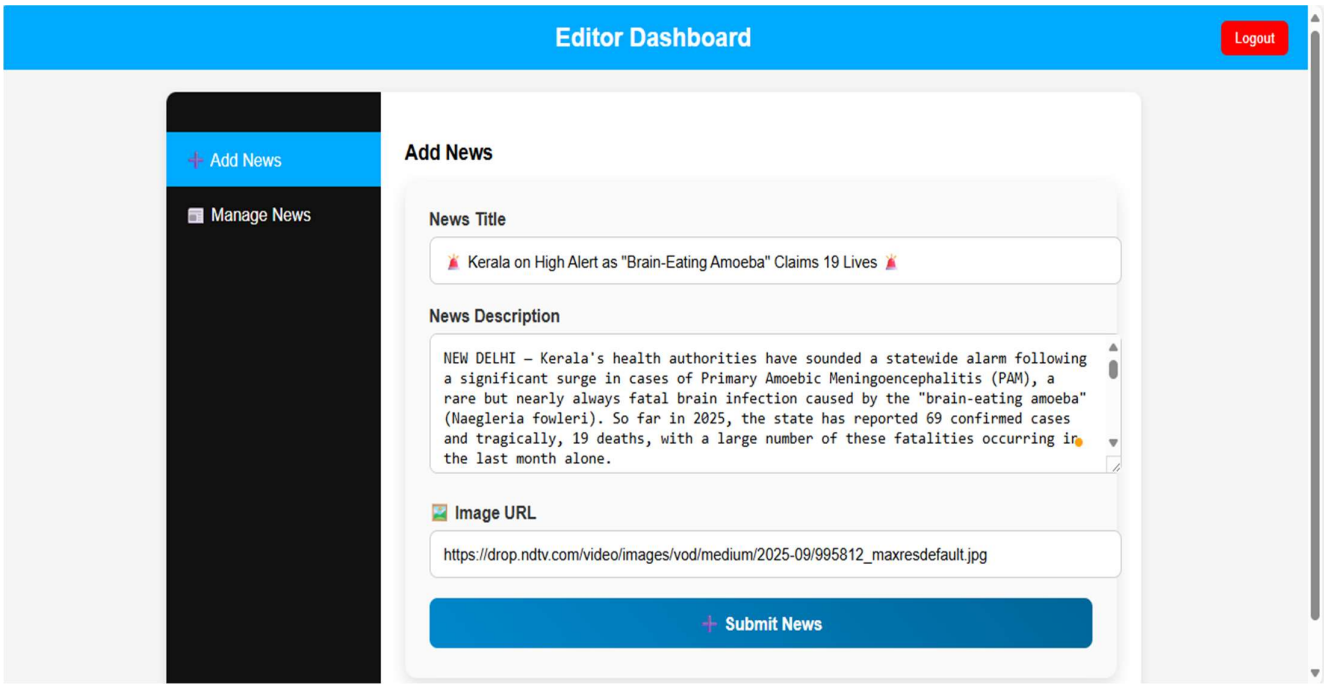
CATEGORY PAGE

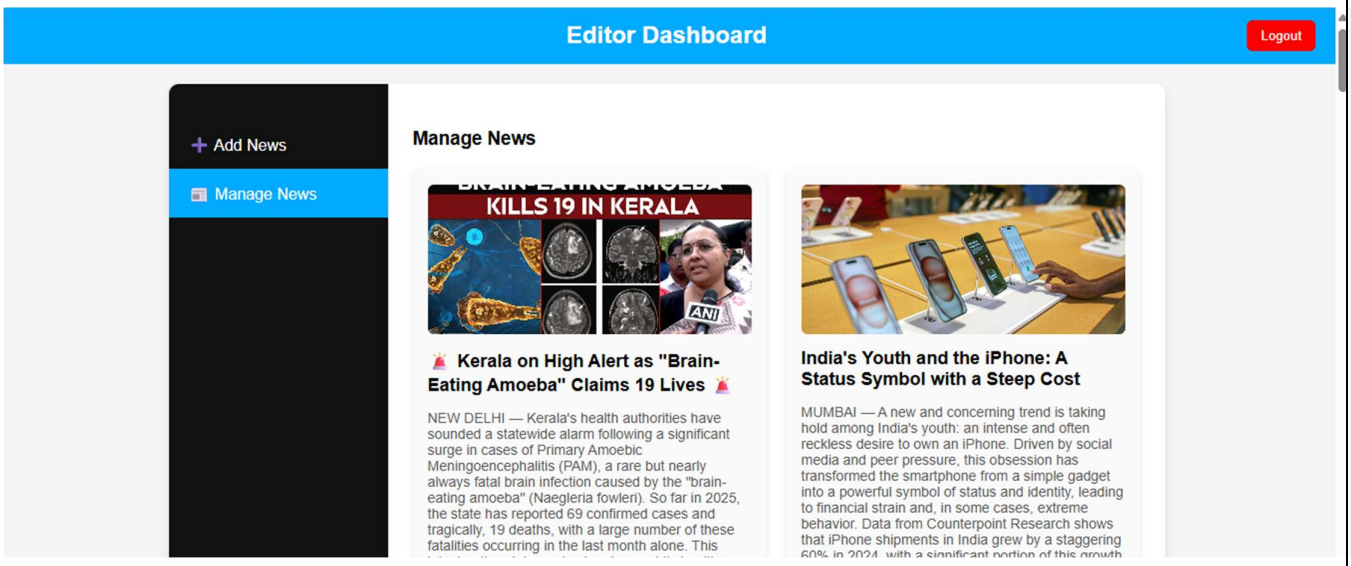


SEARCH BAR

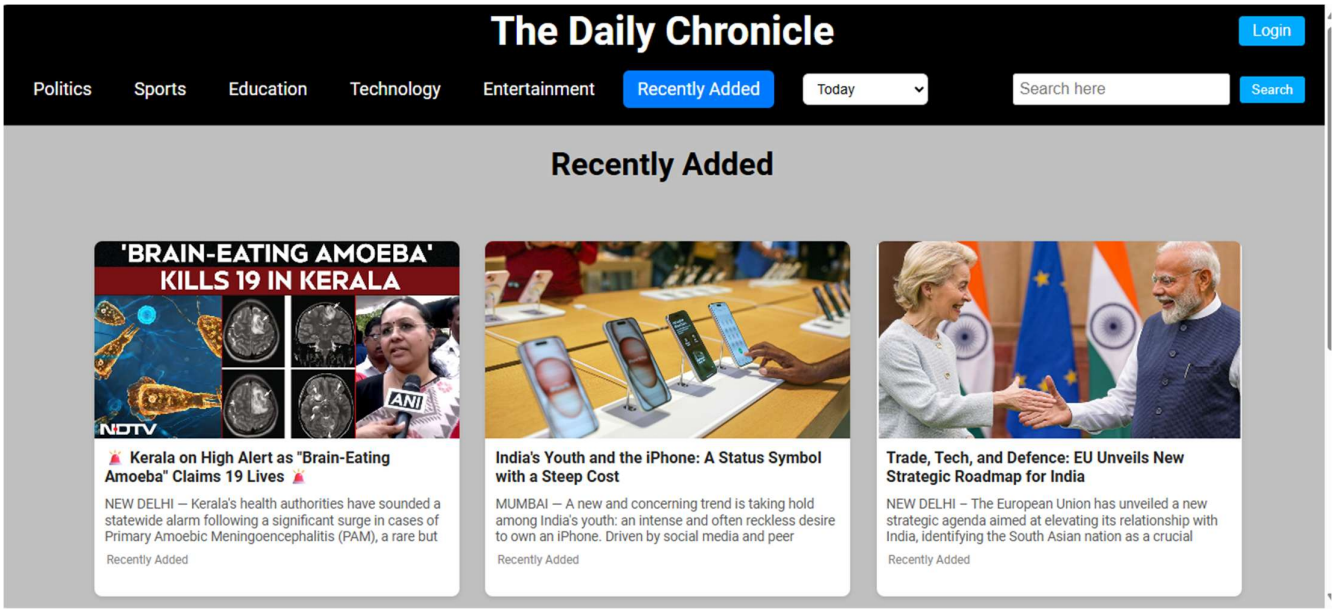


EDITOR'S DASHBOARD

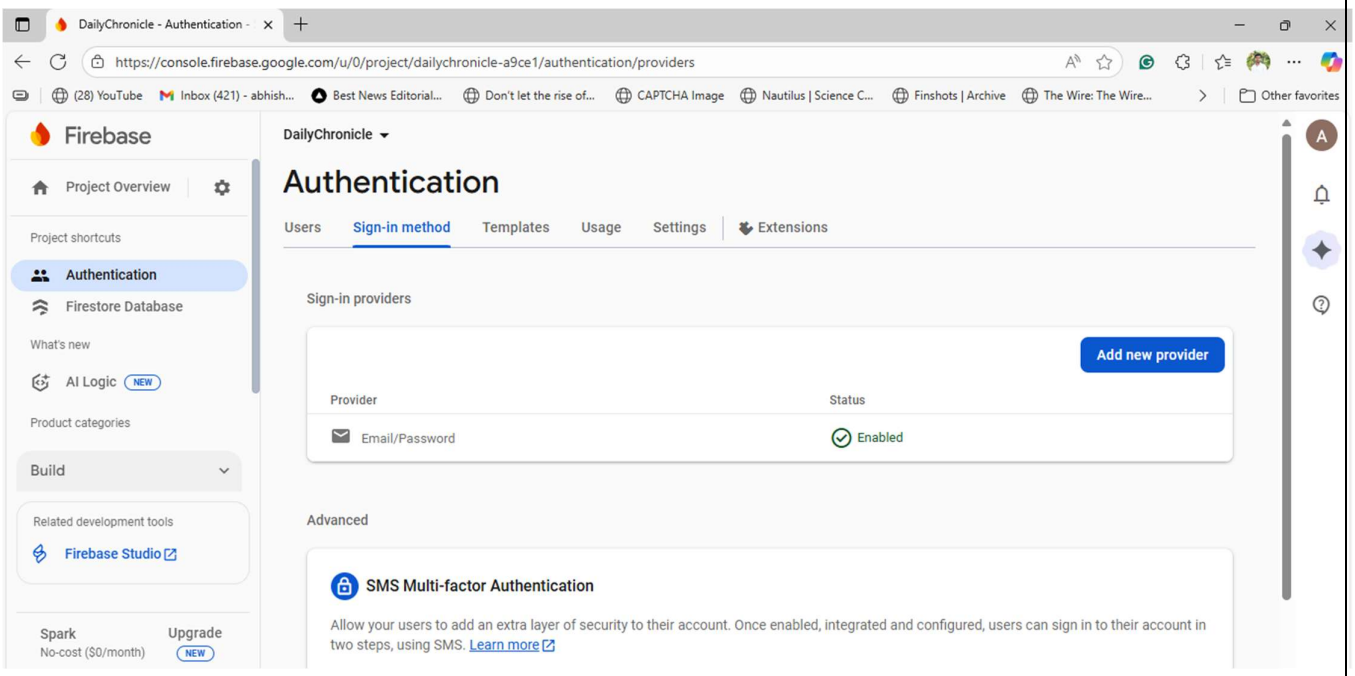




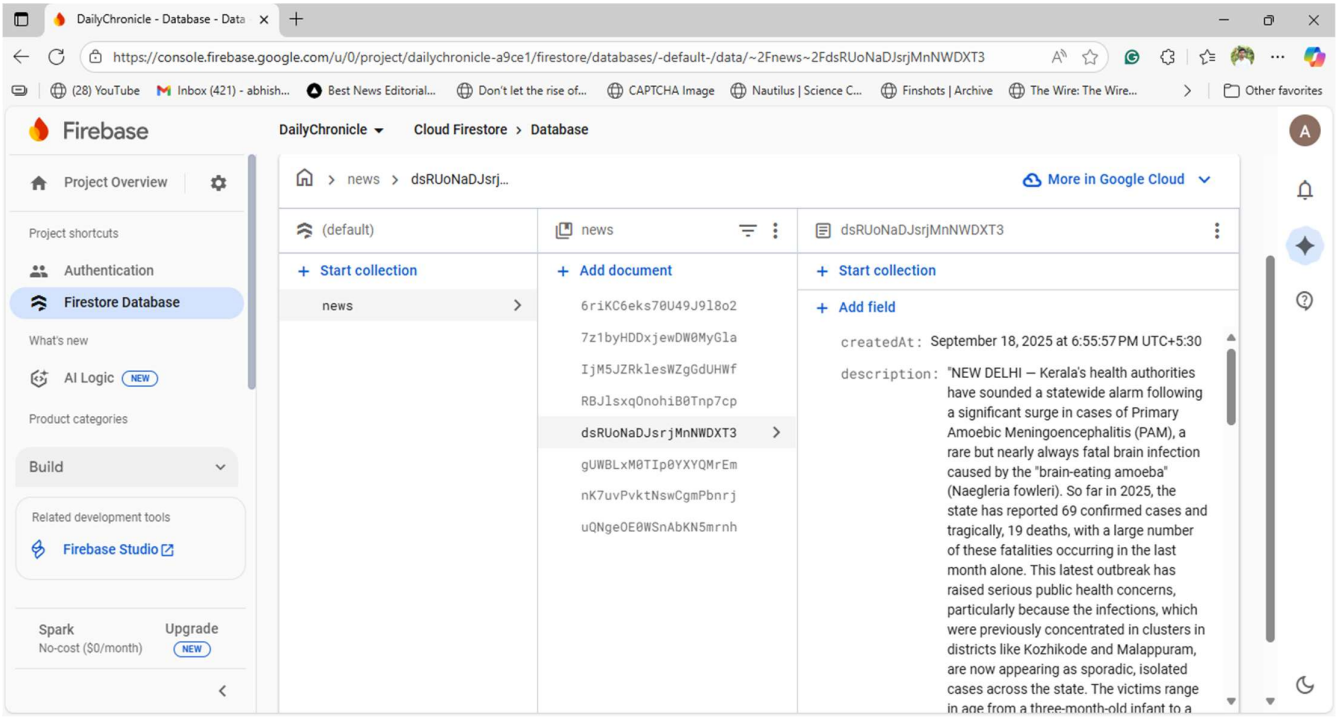
Recently Updated



FIREBASE AUTHENTICATION



FIRESTORE DATABASE



REFERENCES

- www.google.com
- https://www.youtube.com
- https://chatgpt.com